LAB 7: Fuzzing for Input Validation Bugs

Name	Kanishq Surendran
SRN	PES2UG24CS214
Course	Software Engineering — Lab 7 (Fuzz Testing with Hypothesis)
Assigned repo	SETAPESU26/L6_L7_26_loan_eligibility (Lab 7 folder)
Files	loan_eligibility.py (application), test_loan_eligibility.py (Hypothesis tests)
Tools	Python 3, hypothesis 6.170.15, pytest 9.1.1

Objective

Use property-based fuzz testing (Hypothesis) to find input validation bugs in the assigned application **loan_eligibility.py**, which has three functions: **parse_income** ("50,000" → 50000), **monthly_emi** ("principal,rate,years" → EMI) and **pan_category** (PAN number → taxpayer category). The functions are then hardened so that no input can crash them. (In this repo the application file is named loan_eligibility.py instead of processor.py, and the test file is test_loan_eligibility.py instead of test_processor.py.)

Step 1: Installing dependencies

`pip install hypothesis` and `pip install pytest`

Step 2: Running the buggy application

Command: `python loan_eligibility.py`. With the sample inputs the original code works:

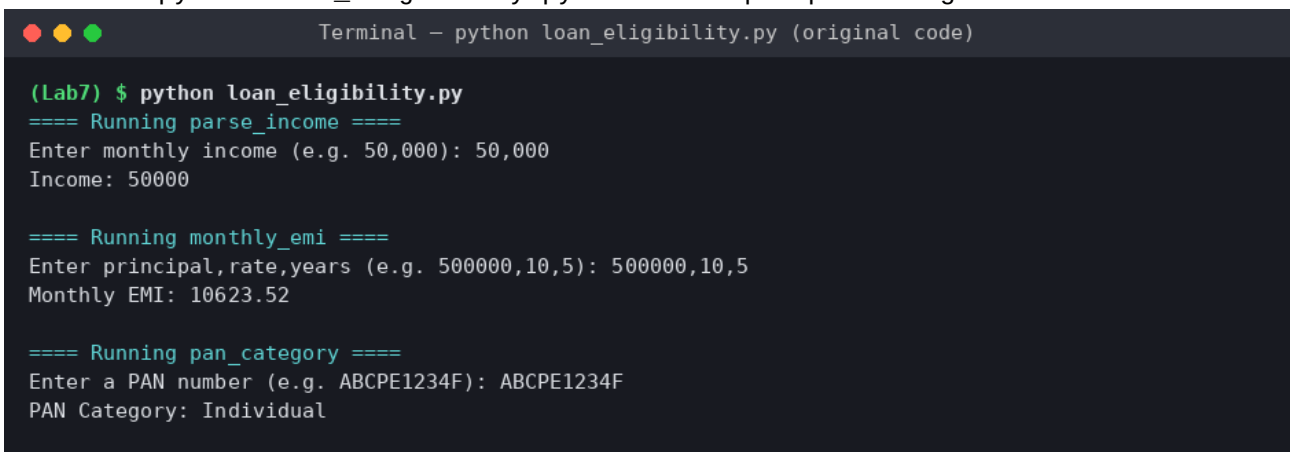


Fig 1: Original application with valid inputs

With a slightly malformed income such as `50k`, the program crashes with an unhandled ValueError:

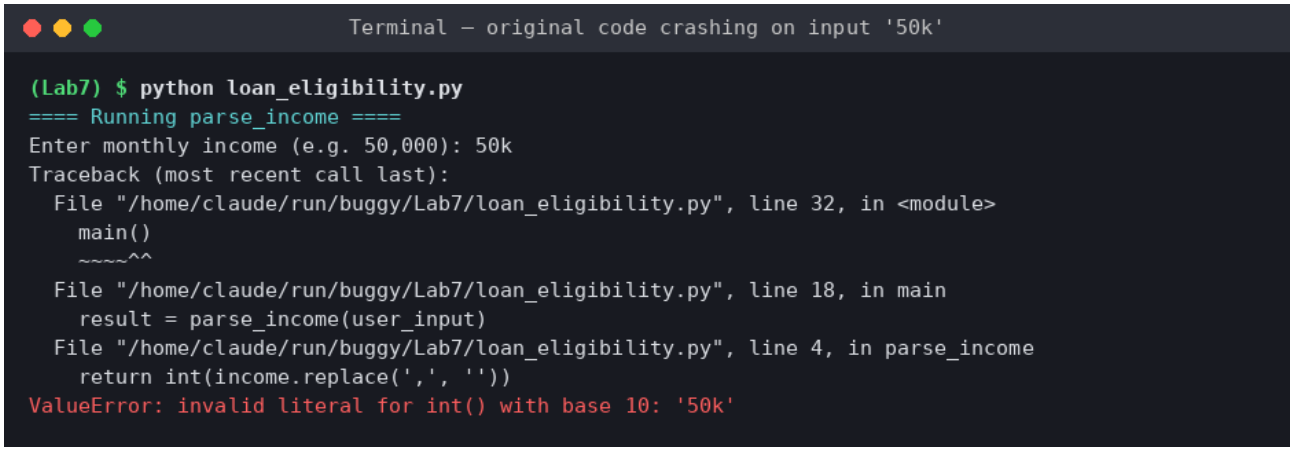


Fig 2: Original application crashing on invalid input

Step 3: Completing test_loan_eligibility.py using Hypothesis

The test file has three kinds of tests, eight in total:
• **No-crash fuzz tests** (one per function): 500 random Unicode strings or None each. Any exception fails the test.
• **Targeted fuzz tests**: "almost valid" inputs, such as comma-separated numbers with empty parts, negatives and huge values, or PAN-like strings of capital letters and digits of the wrong length.
• **Property tests on valid input**: these check that the fix did not break correct behaviour. f"{n:,}" must parse back to n, the EMI for a valid loan must be positive and repay the principal, and a well-formed PAN must map to the right category, in upper or lower case.

The complete test code is in the Appendix.

Step 4: Running the tests on the buggy code

Command: `python -m pytest test_loan_eligibility.py -v`. Result: **5 failed, 3 passed**. All five fuzz tests fail. The three valid-input property tests pass, which confirms the original logic is right for well-formed input.

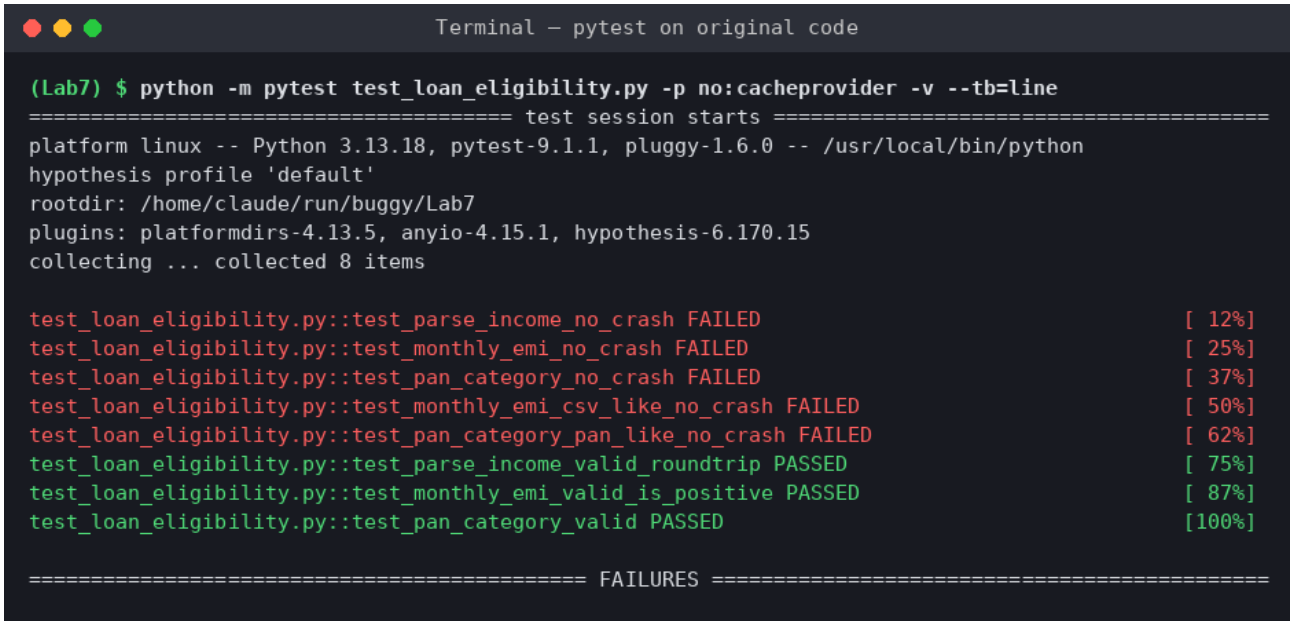


Fig 3: pytest run on the original code: fuzz tests failing

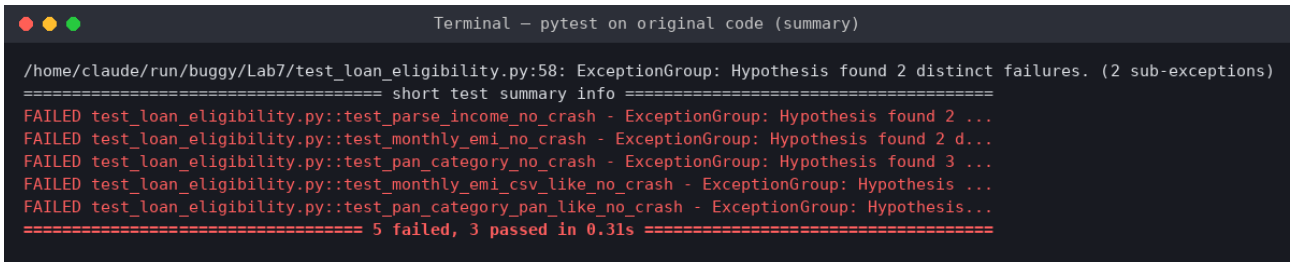


Fig 4: pytest short test summary: 5 failed, 3 passed

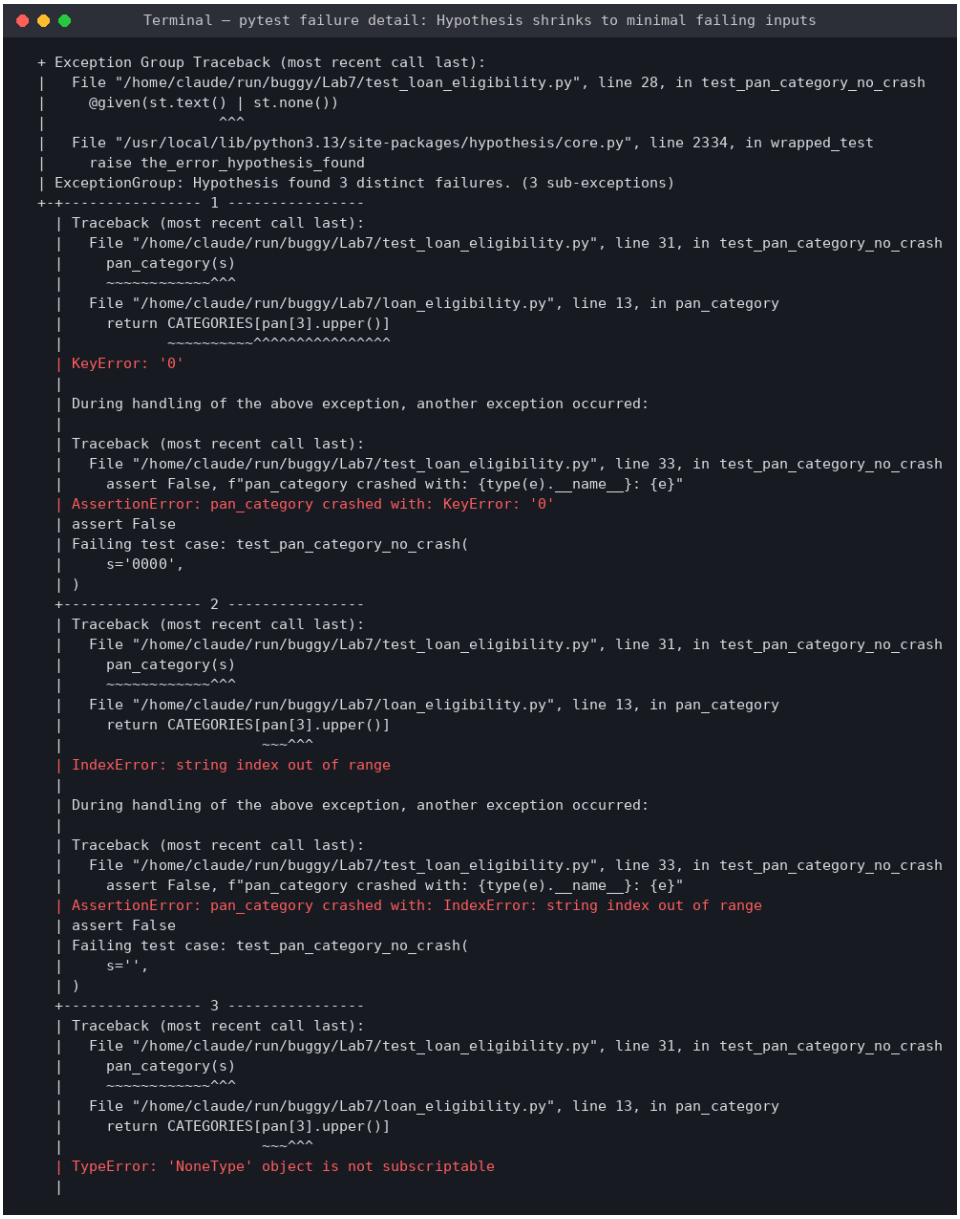


Fig 5: Failure detail for pan_category. Hypothesis shrinks each bug to a minimal failing input ('0000', '', None)

Bugs found by fuzzing

Function	Minimal failing input	Crash / wrong behaviour	Root cause
parse_income	`None`	AttributeError	No type check before .replace()
parse_income	`'' or '50k'`	ValueError	int() on non-numeric text
parse_income	`'-5000', '1_000'`	Accepted silently	int() allows sign and underscores
monthly_emi	`None`	AttributeError	No type check before .split()
monthly_emi	`'' / '1,2'`	ValueError (unpack)	Assumes exactly 3 fields
monthly_emi	`',,' / ',0,0'`	ValueError	Empty fields passed to int()
monthly_emi	`',<huge>,'`	OverflowError	No upper bound on numbers
monthly_emi	`'500000,0,5'`	ZeroDivisionError	0% rate makes the denominator 0
monthly_emi	`'500000,10,0'`	ZeroDivisionError	0 years makes n = 0
monthly_emi	`'-500000,10,5'`	Negative EMI	No range validation
pan_category	`None`	TypeError	No type check before indexing
pan_category	`''`	IndexError	pan[3] on a string shorter than 4
pan_category	`'0000'`	KeyError	4th char not in CATEGORIES

Step 5: Fixing the application

Each function now validates its input defensively and returns **None** for anything invalid instead of raising. main() prints a friendly "Invalid ..." message.

• **Type checks**: anything that is not a str returns None.

• **Strict formats** with regular expressions (ASCII only): income is digits with optional Indian or international comma grouping, EMI needs exactly three plain non-negative integers, and a PAN must match `[A-Z]{5}[0-9]{4}[A-Z]` (case-insensitive, surrounding spaces trimmed).

• **Range checks and limits**: 0 < principal ≤ 10^{12}, 0 ≤ rate ≤ 100, 0 < years ≤ 100, income ≤ 10^{12}. This stops overflows and negative EMIs.

• **0% interest** is handled separately (EMI = principal / months) instead of dividing by zero.

• **Unknown PAN category letters** use CATEGORIES.get() instead of an indexed lookup that raises KeyError.

Running the fixed application: `python loan_eligibility.py`

```
Terminal — python loan_eligibility.py (fixed code)

(Lab7) $ python loan_eligibility.py
==== Running parse_income ====
Enter monthly income (e.g. 50,000): 50,000
Income: 50000

==== Running monthly_emi ====
Enter principal,rate,years (e.g. 500000,10,5): 500000,10,5
Monthly EMI: 10623.52

==== Running pan_category ====
Enter a PAN number (e.g. ABCPE1234F): ABCPE1234F
PAN Category: Individual
```

Fig 6: Fixed application: valid input gives the same results as before

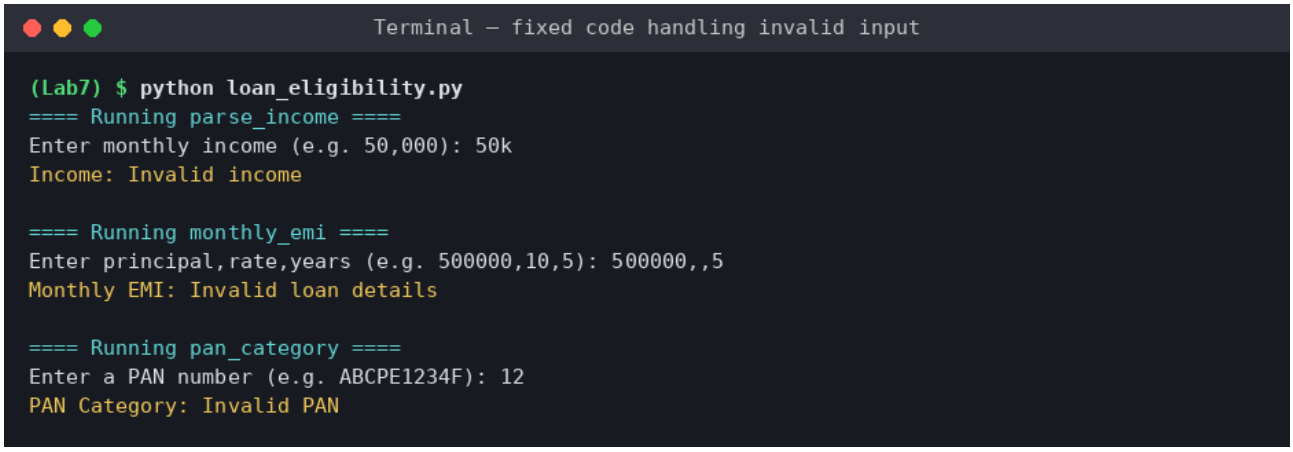


Fig 7: Fixed application: invalid input is rejected gracefully, no crash

Step 6: Re-running the tests on the fixed code

Command: `python -m pytest test_loan_eligibility.py -v`. Result: **8 passed**. No random, malformed or None input crashes any function, and valid inputs still give correct results.

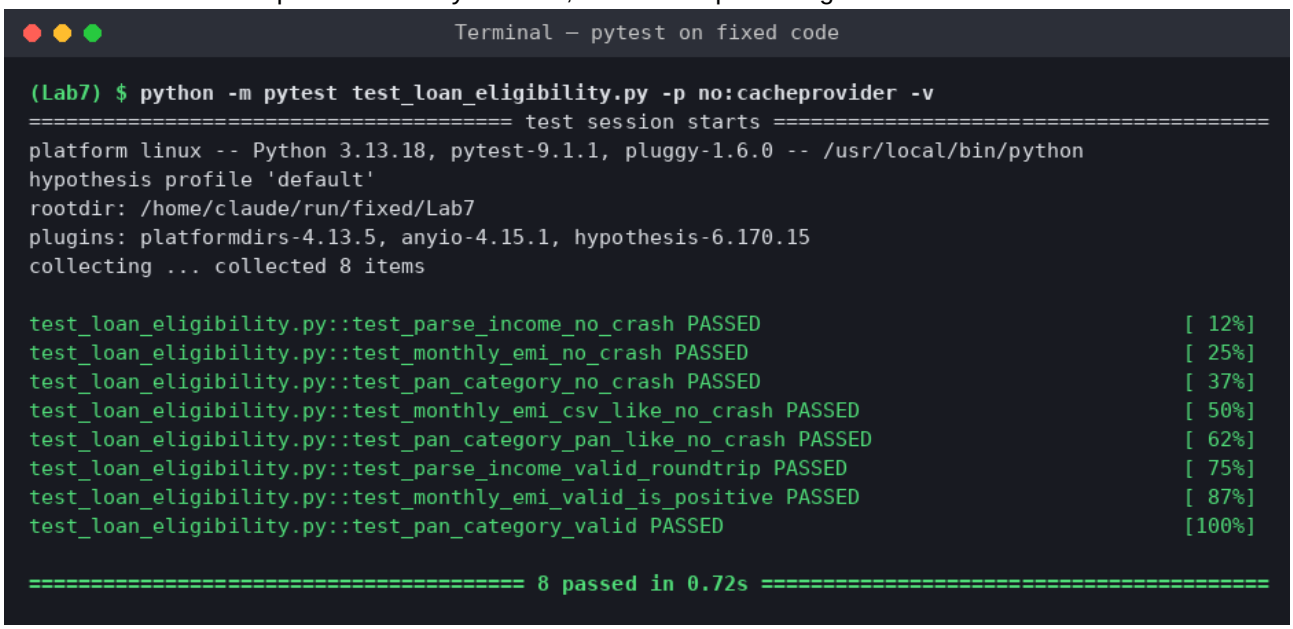


Fig 8: pytest run on the fixed code: all 8 tests pass

Conclusion

Hypothesis found 13 distinct input validation bugs that a few hand-written unit tests would easily miss. These included None inputs, empty fields, oversized numbers, 0% interest and too-short PANs. Shrinking reduced each one to a minimal reproducible input. After the defensive fixes, all fuzz and property tests pass.

Appendix A: Fixed loan_eligibility.py

```
import re

CATEGORIES = {"P": "Individual", "C": "Company", "H": "HUF", "F": "Firm"}

# Sane upper limits so huge inputs can't overflow floats or hang the program
MAX_INCOME = 10**12        # 1 lakh crore
MAX_PRINCIPAL = 10**12
MAX_RATE = 100             # % per annum
MAX_YEARS = 100

# Indian / international grouping or plain digits: 50000, 50,000, 5,00,000
INCOME_PATTERN = re.compile(r"^(\d+|\d{1,3}(,\d{2,3})+)$")
INT_PATTERN = re.compile(r"^\d+$")
PAN_PATTERN = re.compile(r"^[A-Z]{5}[0-9]{4}[A-Z]$")


def _to_positive_int(text):
    """Return int(text) for plain ASCII digits, else None."""
    text = text.strip()
    if not text.isascii() or not INT_PATTERN.fullmatch(text):
        return None
    return int(text)


def parse_income(income):
    """'50,000' -> 50000. Returns None for invalid input instead of crashing."""
    if not isinstance(income, str):
        return None
    income = income.strip()
    if not income.isascii() or not INCOME_PATTERN.fullmatch(income):
        return None
    value = int(income.replace(",", ""))
    if value > MAX_INCOME:
        return None
    return value


def monthly_emi(details):
    """'principal,rate,years' -> EMI rounded to 2 decimals, or None if invalid."""
    if not isinstance(details, str):
        return None
    parts = details.split(",")
    if len(parts) != 3:
        return None
    principal, rate, years = (_to_positive_int(p) for p in parts)
    if principal is None or rate is None or years is None:
        return None
    if not (0 < principal <= MAX_PRINCIPAL):
        return None
    if not (0 <= rate <= MAX_RATE):
        return None
    if not (0 < years <= MAX_YEARS):
        return None

    n = years * 12
    if rate == 0:                       # 0% loan: avoid divide-by-zero
        return round(principal / n, 2)
    r = rate / 1200
    growth = (1 + r) ** n
    return round(principal * r * growth / (growth - 1), 2)


def pan_category(pan):
    """'ABCPE1234F' -> 'Individual'. Returns None for invalid / unknown PANs."""
    if not isinstance(pan, str):
        return None
    pan = pan.strip()
    if not pan.isascii():
        return None
    pan = pan.upper()
    if not PAN_PATTERN.fullmatch(pan):
        return None
    return CATEGORIES.get(pan[3])
def main():
```

```
    print("==== Running parse_income ====")
    user_input = input("Enter monthly income (e.g. 50,000): ")
    result = parse_income(user_input)
    print("Income:", result if result is not None else "Invalid income")

    print("\n==== Running monthly_emi ====")
    user_input = input("Enter principal,rate,years (e.g. 500000,10,5): ")
    result = monthly_emi(user_input)
    print("Monthly EMI:", result if result is not None else "Invalid loan details")

    print("\n==== Running pan_category ====")
    user_input = input("Enter a PAN number (e.g. ABCPE1234F): ")
    result = pan_category(user_input)
    print("PAN Category:", result if result is not None else "Invalid PAN")


if __name__ == "__main__":
    main()
```

Appendix B: test_loan_eligibility.py

```
from hypothesis import given, settings, strategies as st
from loan_eligibility import parse_income, monthly_emi, pan_category, CATEGORIES


# ---------------------------------------------------------------------------
# 1. Robustness (fuzz) tests: random / malformed / None input must never crash
# ---------------------------------------------------------------------------

@settings(max_examples=500)
@given(st.text() | st.none())
def test_parse_income_no_crash(s):
    try:
        parse_income(s)
    except Exception as e:
        assert False, f"parse_income crashed with: {type(e).__name__}: {e}"


@settings(max_examples=500)
@given(st.text() | st.none())
def test_monthly_emi_no_crash(s):
    try:
        monthly_emi(s)
    except Exception as e:
        assert False, f"monthly_emi crashed with: {type(e).__name__}: {e}"


@settings(max_examples=500)
@given(st.text() | st.none())
def test_pan_category_no_crash(s):
    try:
        pan_category(s)
    except Exception as e:
        assert False, f"pan_category crashed with: {type(e).__name__}: {e}"


# ---------------------------------------------------------------------------
# 2. Targeted fuzzing: "almost valid" inputs that look like real user input
# ---------------------------------------------------------------------------

# Comma separated numbers, including empty parts, negatives, extra/missing parts
number_like = st.one_of(
    st.integers(min_value=-10**6, max_value=10**6).map(str),
    st.just(""),
    st.just(" "),
    st.integers(min_value=0, max_value=10**400).map(str),
)


@given(st.lists(number_like, min_size=0, max_size=5).map(",".join))
def test_monthly_emi_csv_like_no_crash(s):
    try:
        monthly_emi(s)
    except Exception as e:
        assert False, f"monthly_emi crashed on {s!r} with: {type(e).__name__}: {e}"


@given(st.text(alphabet="ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789", max_size=12))
def test_pan_category_pan_like_no_crash(s):
    try:
        pan_category(s)
    except Exception as e:
        assert False, f"pan_category crashed on {s!r} with: {type(e).__name__}: {e}"


# ---------------------------------------------------------------------------
# 3. Property tests: valid input must still give the correct answer
# ---------------------------------------------------------------------------

@given(st.integers(min_value=0, max_value=10**9))
def test_parse_income_valid_roundtrip(n):
    assert parse_income(f"{n:,}") == n      # "50,000" -> 50000
    assert parse_income(str(n)) == n        # "50000"  -> 50000
@given(
    st.integers(min_value=1000, max_value=10**8),   # principal
```

```
    st.integers(min_value=1, max_value=30),      # annual rate %
    st.integers(min_value=1, max_value=30),      # years
)
def test_monthly_emi_valid_is_positive(p, rate, years):
    emi = monthly_emi(f"{p},{rate},{years}")
    assert emi is not None and emi > 0
    # total paid back must be at least the principal
    assert emi * years * 12 >= p - 1


@given(
    st.text(alphabet="ABCDEFGHIJKLMNOPQRSTUVWXYZ", min_size=3, max_size=3),
    st.sampled_from(sorted(CATEGORIES)),
    st.text(alphabet="ABCDEFGHIJKLMNOPQRSTUVWXYZ", min_size=1, max_size=1),
    st.integers(min_value=0, max_value=9999),
    st.text(alphabet="ABCDEFGHIJKLMNOPQRSTUVWXYZ", min_size=1, max_size=1),
)
def test_pan_category_valid(prefix, cat, fifth, digits, last):
    pan = f"{prefix}{cat}{fifth}{digits:04d}{last}"
    assert pan_category(pan) == CATEGORIES[cat]
    assert pan_category(pan.lower()) == CATEGORIES[cat]
```